

Анонимные функции

- Введение в анонимные функции
- Замыкания (closures)
- Стрелочные функции
- Функции обратного вызова
- Подведем итоги

Введение в анонимные функции

Анонимные функции позволяют создавать функции, не имеющие определенных имен. Синтаксис определения анонимной функции не отличается от синтаксиса определения обычной функции за тем исключением, что она не имеет имени.

Шаблон определения анонимной функции:

```
<?php  
  
function () {  
  
    // тело функции  
  
}
```

Анонимные функции наиболее полезны в качестве **значений параметров** других функций, но могут иметь и множество других применений.

Анонимные функции могут быть использованы в качестве **значений переменных**. В таком случае присвоение переменной использует тот же синтаксис, что и любое другое присвоение, включая завершающую точку с запятой.

Для вызова подобной функции применяется имя представляющей ее переменной.

example_1. Пример анонимной функции

```
<?php

    // присвоение переменной анонимной функции

    $greet = function() {

        echo "Привет, давайте изучать PHP вместе!";

    };

    $greet();

?>
```

Фактически подобная **переменная** применяется как **стандартная функция**.

Также анонимные функции могут **возвращать** некоторое значение:

```
<?php

    $greet = function () {

        return "Привет, давайте изучать PHP вместе!";

    };

    echo $greet();
```

Анонимные функции могут принимать обычные **параметры**.

example_2. Передача параметров анонимной функции

```
<?php

    // присвоение переменной анонимной функции

    // функции передаем параметр

    $greet = function($par) {

        printf("Привет, давайте изучать %s вместе!</p>", $par);

    };

    // иницилируем значения аргументов анонимной функции
```

```
$lang1 = "PHP";

$lang2 = "JavaScript";

// вызываем функцию

$greet($lang1);

$greet($lang2);

?>
```

Параметры анонимных функций могут передаваться как **по значению**, так и **по ссылке**.

example_3. Передача параметра по ссылке

```
<?php

// присвоение переменной анонимной функции

// функции передаем параметр по ссылке

$greet = function(&$par) {

    $par = "Виноделие";

    printf("Привет, давайте изучать %s вместе!</p>", $par);

};

// иницируем значения аргументов анонимной функции

$lang1 = "PHP";

$lang2 = "JavaScript";

// вызываем функцию

$greet($lang1);

// $lang1 передавали в функцию

echo '$lang1 = ' . $lang1 . "<p>";

// $lang2 - нет

echo '$lang2 = ' . $lang2;

// передайте параметр $par анонимной функции по значению и
```

```
протестируйте еще раз
```

```
?>
```

Кроме того, как и в обычных функциях, в анонимных функциях может использоваться ключевое слово **global**.

example_4. Использование глобальных переменных

```
<?php

// иницилируем переменную $role

$role = "Администратор";

$test = function() {

    // делаем переменную глобальной

    global $role;

    var_dump("Вы зашли как " . $role . " сайта.");

};

$test();

?>
```

Замыкания (closures)

Замыкания в PHP представляют анонимную функцию, которая может **наследовать переменные** из родительской области видимости. Любая подобная переменная должна быть объявлена в конструкции **use**.

example_5. Наследование глобальных переменных

```
<?php

$role = "Администратор";

// наследуем переменную role

$example = function () use($role) {
```

```
        var_dump("Вы зашли как " . $role . " сайта.");  
  
    };  
  
    $example();  
  
?>
```

Выражение **use** получает **внешние переменные родительской области видимости**, которые анонимная функция собирается использовать.

Внешние переменные в замыканиях могут передаваться как по **значению**, так и по **ссылке**.

example_6. Наследование по ссылке

```
<?php  
  
    $role = "Администратор";  
  
    echo '$role = ' . $role . "<p>";  
  
    // наследуем переменную role  
  
    $example = function () use(&$role) {  
  
        $role = "Модератор";  
  
    };  
  
    $example();  
  
    echo '$role = ' . $role . "<p>";  
  
?>
```

К сведению. Наследование переменных из глобальной области видимости в замыканиях **не то же самое**, что использование **глобальных** переменных.

Разница между **наследованием** переменной из глобальной видимости и использованием ключевого слова **global** наглядно продемонстрирована в коде листинга файла example_7.php.

example_7. Наследуемые и глобальные переменные

```
<?php

// иницилируем переменную $role

$role = "Администратор";

// определяем функцию, наследуем переменную role

// !!! обратите внимание -

// test_use была определена, когда $role = Администратор

$test_use = function () use($role) {

    var_dump("Вы зашли как " . $role . " сайта.");

};

// определяем функцию, делаем переменную role глобальной

$test_global = function () {

    global $role;

    var_dump("Вы зашли как " . $role . " сайта.");

};

// переопределим переменную $role

$role = "Модератор";

$test_use(); // Вы зашли как Администратор сайта

echo "<p>";

$test_global(); // Вы зашли как Модератор сайта

?>
```

Важно. Значение унаследованной переменной задано там, где

функция **определена**, но не там, где **вызвана**.

Подобным образом функция-замыкание может захватывать и **большее количество внешних переменных**.

example_8. Наследуемых переменных может быть много

```
<?php

    // иницилируем переменные

    $phone = "2-12-85-06";

    $role = "Администратор";

    // передаем параметры, наследуем переменные

    $message = function ($login, $pwd) use ($role, $phone) {

        echo "<pre>";

        var_dump($login, $pwd, $role, $phone);

    };

    // вызываем функцию с аргументами

    $message("master", "12345");

?>
```

Ну, и естественно, можно сочетать все эти способы работы с переменными в анонимных функциях

example_9. Одержимым шпиономанией

```
<?php

    // иницилируем переменные

    $phone = "2-12-85-06";

    $role = "Администратор";

    // передаем параметры, наследуем переменные
```

```
$message = function ($login, $pwd) use ($role) {  
  
    // получаем доступ  
  
    global $phone;  
  
    echo "<pre>";  
  
    var_dump($login, $pwd, $role, $phone);  
  
};  
  
// вызываем функцию с аргументами  
  
$message("master", "12345");  
  
?>
```

Объявление **типа возвращаемого значения** функции может быть помещено после конструкции **use**.

```
<?php  
  
$user = "Batman";  
  
$example = function () use ($user): string {  
  
    return "Привет mr. $user";  
  
};  
  
echo $example();
```

Стрелочные функции

Стрелочные функции (arrow function) появились в PHP 7.4, как более лаконичный синтаксис для анонимных функций, которые возвращают некоторое значение. И при этом стрелочные функции **автоматически** имеют доступ к переменным из **внешнего** окружения.

Стрелочная функция определяется с помощью оператора **fn**:

```
fn (параметры) => действия;
```


example_10. Стрелочная функция

```
<?php

    // инициализация переменных

    $a = 8;

    $b = 10;

    // определение стрелочной функции

    $fun = fn() => $a + $b;

    // вызов функции-стрелки

    echo $fun(); // 18

?>
```

Вот некоторые особенности использования стрелочных функций в PHP:

- становятся доступными в PHP 7.4;
- начинаются с ключевого слова **fn**;
- могут иметь только **одно** выражение, которое являет собой **return** выражение, при этом ключевое слово **return** указывать **не** нужно;
- функциям могут задаваться возвращаемые типы.

Ниже представлен пример использования анонимной функции и аналогичной ей стрелочной.

example_11. Анонимная функция против стрелочной

```
<?php

    $y = 5;

    // определение и присвоение переменной анонимной функции

    $fun1 = function ($x) use ($y) {

        return $x + $y;

    };

    // вызов анонимной функции
```

```

var_dump ($fun1(4)); // 9

// определение и присвоение переменной стрелочной функции
$fun2 = fn($x) => $x + $y;

echo "<p>";

// вызов стрелочной функции
var_dump ($fun2(4)); // 9

?>

```

Разработчик с опытом программирования в JavaScript, может попробовать описать стрелочную функцию указывая несколько выражений.

```

$fun = fn() => {
    // ...

    return 1;
}

```

Но, в PHP так делать нельзя.

Важно. Стрелочная функция описывается в одну строку, использовать несколько строк для её описания **нельзя**.

Стрелочные функции используют привязку переменных **по значению**. Привязка по значению означает, что невозможно изменить какие-либо значения из внешней области. Вместо этого можно использовать анонимные функции для привязок по ссылкам.

example_12. Привязка переменных в стрелочных функциях

```

<?php

$x = 5;

// ничего не изменит

```

```
$fn = fn() => $x++;  
  
$fn();  
  
echo $x; // 5  
  
?>
```

Функции обратного вызова

Распространенным случаем применения анонимных функций является передача их параметрам **других функций**. Такие анонимные функции называют **функциями обратного вызова** или коллбеками (callback function).

Важно. Функция, переданная в качестве параметра другой функции, называется **функцией обратного вызова** (callback).

Функции обратного вызова достаточно объемный материал для изучения при программировании пользовательских функций.

Рассмотрим принцип действия callback-функций на примере работы стандартной функции **array_map** (урок Встроенные функции 3).

Функция `array_map` служит для **итеративной** обработки элементов массива. Callback-функция применяется к **каждому элементу массива** и в качестве результата выдается обработанный массив.

```
array_map(callback, $array);
```

Параметр

- **callback** - функция, применяемая к каждому элементу в каждом массиве.
- **array** - массив, к каждому элементу которого применяется функция.

example_13. Функция array_map

```
<?php

    // иницилируем исходный массив

    $nums = array(1, 2, 3, 4, 5);

    // в качестве параметра передаем анонимную функцию
    // функция возводит в квадрат переданный параметр

    $result = array_map(function ($num) {

        return $num * $num;

        // без return результирующий массив будет пустой

    }, $nums);

    // выводим результат

    echo "<pre>";

    print_r($result); // 1,4,9,16,25

?>
```

В коде листинга example_14 приведен другой вариант использования callback-функции.

example_14. Определение callback-функции

```
<?php

    // иницилируем исходный массив

    $nums = array(1, 2, 3, 4, 5);

    // определяем функцию

    $fnSquare = function ($num) {

        return $num * $num;

    };

    // в качестве параметра передаем callback-функцию

    // функция возводит в квадрат переданный параметр
```

```
$result = array_map($fnSquare, $nums);

// выводим результат

echo "<pre>";

print_r($result); // 1,4,9,16,25

?>
```

Ну, и не забываем, анонимная функция может, как **наследовать**, так и получать **доступ** к переменным глобального уровня видимости.

example_15. Наследование в анонимных функциях

```
<?php

// иницилируем исходный массив

$tags = array("php", "laravel", "wordpress", "mysql",
"free");

// внешняя переменная

$grid = "#";

// определяем callback-функцию

$fnSquare = function ($key) use($grid) {

    return $grid . $key;

};

// в качестве параметра передаем callback-функцию

$result = array_map($fnSquare, $tags);

// выводим результат

echo "<pre>";

print_r($result); // конечный массив

?>
```

Подведем итоги

Функция - это специальным образом оформленный именованный фрагмент кода, к которому можно многократно обратиться из любого места внутри программы. Функция не будет выполняться сразу при загрузке страницы. Функция будет выполняться при ее **вызове** из тела программы.

Информация может передаваться внутрь функции:

- через **параметры** (могут передаваться по **значению** и по **ссылке**);
- через ключевое слово **global**.

Определение пользовательской функции задаёт **локальную** область видимости данной функции. Любая используемая внутри функции переменная по умолчанию ограничена локальной областью видимости функции. К таким переменным можно обратиться только **изнутри данной функции**.

Функция может принимать в качестве параметров и возвращать в качестве результата своей работы любой используемый в PHP тип данных.

Если функция не использует оператор **return**, она все равно возвращает значение, только в этом случае это значение - **Null**.

PHP позволяет создавать **анонимные функции**, такие функции не имеют определенных имен. Синтаксис определения анонимной функции не отличается от синтаксиса определения обычной функции за тем исключением, что она не имеет имени.

Анонимные функции наиболее полезны в качестве **значений параметров** других функций.

Анонимные функции, которые могут наследовать переменные из родительской области видимости, называют **замыканиями**.

Анонимная функция может принимать **внешние переменные**:

- через параметры (по значению, по ссылке);
- через наследование (по значению, по ссылке);
- глобально.

Стрелочная функция - краткая форма записи анонимной функции (имеет особенность - прямой доступ к глобальным переменным)

Функции обратного вызова - анонимные функции, переданные в качестве **параметра** другой функции.